Pre-Meeting Brief Platform — Approach

Pre-Meeting Brief Platform — Approach
- Assignment — verbatim re-read
  - How this document addresses each workflow component
  - Architectural choices justified from the primary brief alone
  - Scope expansions beyond the four-component spec (and why)
  - What is still owed against the four components
- 1. Summary
- 2. Reading the Inputs
  - Buried gold — ten things only careful reading surfaces
- 3. Gaps in the Original Architecture
  - A. Operational (resilience, observability)
  - B. Product (UX, content)
  - C. Architecture (system structure)
  - D. Quality / Feedback (process)
- 4. POC Architecture
  - 4.1 POC additions over the original v1 spec
  - 4.2 Production deltas — what real-world deployment swaps in or out
- 5. Agentic Workflow Design
  - 5.1 Qualification Agent
  - 5.2 Research Agent
  - 5.3 Data Quality Agent
  - 5.4 Synthesis Agent
  - 5.5 Deterministic core
  - 5.6 Orchestration
- 6. MCP Server Design
- 7. Data Model
  - 7.1 Mirrors
  - 7.2 Adaptations
  - 7.3 Additions
  - 7.4 Source-priority chains (verbatim from DD)
  - 7.5 Conflict detection rules
  - 7.6 Cross-field consistency rules (rule engine)
- 8. Synthesis & Confidence UX
  - 8.1 System prompt (full template)

8.2 Output JSON schema
8.3 Confidence UX
8.4 Trust boundaries (prompt-injection defense)
8.5 Brief layout (dashboard hybrid)
9. Phase Plan
What we explicitly cut and why
10. Production Roadmap
11. Submission Deliverables
Appendix
A. Six demo companies (selected for thesis-fit variety)
B. Tech stack summary
C. Key project constants

Pre-Meeting Brief Platform — Approach

Author: Rishav Chatterjee **Date**: 2026-05-28 **Submission to**: Capital Numbers — AI-Native Tech Lead role (Gurgaon) **Designed around**: Renegade Capital — the example VC client in the assignment materials **Submission deadline**: 2026-05-29 EOD **Live POC**: https://rishavchatterjee.com/pre-meeting-brief **Repo**: https://github.com/rishav1305/pre-meeting-brief

---

Assignment — verbatim re-read

The email defines the deliverable as a **written approach** that addresses four workflow components. I quote them here so the rest of the document can be measured against them directly:

> 1. **Trigger Mechanism**: How the system detects a new meeting with a company not engaged with in the last three months.
> 2. **Data Integration**: Your strategy for pulling and merging data from CRM (Attio), 3rd party sources (Specter, Crunchbase), and the web.
> 3. **Synthesis**: How you would use LLMs to generate a structured brief that includes demographics, key personnel, and market deep dives.
> 4. **Distribution**: The process for automatically attaching the final brief to a daily agenda.

The email also says: *"Linear Tickets: Please proceed with your written approach based on the primary brief and architecture diagram provided in the main file."* I read this as **Linear Tickets are not assumed accessible to the reviewer evaluating my submission, and architectural choices should be derivable from the primary brief + architecture diagram + Data Dictionary alone**. Where the rest of this document references ticket IDs, they are after-the-fact corroboration of the same conclusions, not the basis for them.

How this document addresses each workflow component

Workflow component (verbatim from email)	Where it is addressed	Status in the POC today
Trigger Mechanism — detect new meeting + 3-month rule	§4 (architecture), §5.1 (Qualification Agent), `GET /api/triggers/scan`	Built (fixture-backed) — Vercel Cron fires daily at 06:00 UTC, walks `canonical.calendar_events` for the next 24h, skips events that already have a brief, kicks the pipeline (which then runs the 90-day qualification gate). Calendar *source* is still fixture-seeded for the POC; OAuth-driven Google Calendar / Outlook integration is the production swap (see §10).
Data Integration — Attio + Specter + Crunchbase + web	§6 (MCP boundary), §7.4 (priority chains), §7.5 (conflict rules)	Built — 4 providers (Attio, Specter, Crunchbase, PitchBook) wired through a common Protocol with deterministic merge + conflict detection. Web context via Anthropic `web_search` tool inside the Research Agent.
Synthesis — LLM-generated brief with demographics + personnel + market	§5.4 (Synthesis Agent), §8 (system prompt, schema, UX)	Built — Sonnet 4.6 forced `tool_use` with draft→critique→revise loop; brief includes Snapshot (demographics), Team (personnel), Market deep-dive and Industry deep-dive sections.
Distribution — attach to daily agenda	§4 (architecture diagram), `api/distribution/calendar_writeback.py`	Built as a logged stub — after `render_and_persist`, the pipeline constructs the exact Google Calendar Events.patch() payload (calendar_id,

Workflow component (verbatim from email)	Where it is addressed	Status in the POC today
		event_id, append-block with brief URL + timestamp) and persists it to `pre_meeting_brief.distribution_log`. A "Distribution → Google Calendar: Payload built (POC mock)" badge renders on the brief reader. The HTTP dispatch is deferred to production-time OAuth (a one-flag flip from `_LOG_ONLY=True` to `False`). Slack/Attio/Gmail channels are roadmap (§10).

Architectural choices justified from the primary brief alone

The reviewer should be able to read just the email and the architecture diagram and arrive at these same calls:

- **A graph orchestrator (not chained function calls)** is needed because the brief's flow fans out across multiple sources (Attio + Specter + Crunchbase + web), merges them, then synthesizes — a stateful DAG with shared `BriefState`, not a single request/response. LangGraph is one credible implementation; Prefect, Temporal, or a hand-rolled async DAG runner would also work. The choice of LangGraph specifically is for ecosystem maturity around LLM tool-call orchestration, not because a ticket prescribed it.
- **An MCP boundary on source providers** is the right abstraction because each source is independently authenticated, independently rate-limited, and the same servers should be reusable from Claude Desktop for ad-hoc partner lookups outside the app. MCP is the emerging standard for this exact shape.
- **A deterministic merge with explicit priority chains** is needed because the Data Dictionary specifies source priority per field. Doing this in an LLM would erode reproducibility.
- **An agentic Synthesis layer with self-critique** is needed because the brief is a judgment artifact — a partner reading it expects calibrated language, sharp questions, and a tight narrative, none of which a single-shot LLM call reliably produces.
- **Conflict detection at merge time** is needed because three sources of structured data will disagree. The DD's source priority resolves

which value wins for the brief; the conflicts themselves are first-class data the partner should see.

Scope expansions beyond the four-component spec (and why)

Addition	Rationale
PitchBook as a fourth source provider	The email lists Specter and Crunchbase; the Data Dictionary's source-priority chains reference PitchBook for funding fields (`total_raised_usd`, `last_round_*`, `post_money_valuation_usd`). I treat the DD as the schema of record. Worth naming the discrepancy explicitly: if PitchBook is unavailable in production, the priority chains gracefully degrade to Specter and Crunchbase.
`thesis_fit { score, reasoning, bear_case }` **in the brief schema**	The brief is for a specific VC client (Renegade) with a stated thesis. A pre-meeting brief that does not score against the firm's thesis is doing half the job. For multi-tenant production, the rubric becomes a per-firm config (see §10 and the `firm_config` work).
`key_engagement_questions` **and** `prior_engagement` **sections**	These come from reading the brief's intent ("refresh memory, unified perspective"). A partner about to walk into a meeting wants the 3-5 sharpest questions to ask and a timeline of prior interactions. The email's "demographics + personnel + market" is the minimum; a useful brief includes both.
`new_highlights` **lead in the Snapshot**	The DD's `traction_metrics` sheet distinguishes `highlights` from `new_highlights`. The latter exist precisely so the brief can answer "what changed since the partner last looked." Treating new signals

Addition	Rationale
	as the lead is a direct consequence of the schema.

What is still owed against the four components

The gap is concentrated in Trigger and Distribution. Both are tracked in `docs/review-gaps.md` and are the next two engineering pushes (`firm_config` parameterization, calendar-distribution stub, Vercel-Cron scan endpoint). The Synthesis and Data Integration components are built end-to-end and demoable on the live URL.

---

1. Summary

This is a take-home submission for the **Capital Numbers AI-Native Tech Lead** role. The assignment brief — a pre-meeting brief automation system — references **Renegade Capital** as the example VC client (their thesis "Markets That Matter," their data dictionary). The platform is therefore *designed for* Renegade and *delivered to* Capital Numbers as the architecture-and-build deliverable. Throughout this document, "the firm" refers to Renegade as the demo client; "the reviewer" refers to Capital Numbers' hiring panel.

Build an automated pre-meeting brief platform for Renegade Capital partners. Triggered on the first meeting with a company within a rolling 3-month window. Pulls structured data from CRM (Attio) and third-party sources (Specter, Crunchbase, PitchBook), fetches live web context via Anthropic's `web_search` tool, merges with explicit source-priority rules from the data dictionary, and runs an agentic synthesis loop tuned to Renegade's "Markets That Matter" thesis. The resulting brief — a tiered dashboard with confidence-marked facts and a click-through audit panel — is surfaced on the partner's daily agenda.

Three design moves the original spec did not make:

1. **Agentic where judgment helps, deterministic where consistency matters.** Agents drive the web research loop, synthesis reflect-and-revise, qualification edge cases, and data-quality triage. Domain normalization, the merge priority chains, DB writes, and rendering stay deterministic. The original spec is a heuristic DAG; ours puts agency in four specific places where it produces measurably better output, and keeps determinism everywhere it would erode trust.
2. **Source providers exposed as MCP servers.** Each external source (Specter, Crunchbase, PitchBook, Attio) is an independent MCP

server. The orchestrator calls them as tools. Pluggable: fixture-backed today, real-API-backed tomorrow, zero orchestrator changes. Same servers can be connected to Claude Desktop for ad-hoc lookups outside the app.
3. **Confidence-first UX.** Every fact in the rendered brief carries a confidence dot. Click → audit panel slides in with source-by-source attribution. Data quality conflicts surface as flagged badges. The DD's `audit VARIANT` pattern becomes a visible product feature, not just a logging detail.

Demo flow at the live URL:

- Land on `/pre-meeting-brief` → see Daily Agenda with tomorrow's qualifying first-meetings as preview cards.
- Click "Anduril Industries" → full brief dashboard: Thesis Fit (5/5 + reasoning + bear case), Snapshot, Funding, Team, Traction, Prior Engagement timeline, Industry/Market deep-dives, Key Questions, Media. Confidence dots inline; click any → audit panel.
- Visit `/pre-meeting-brief/admin` (password-gated), enter `mercury.com` → watch the Synthesis Agent run live in ~30s with visible tool calls and a draft-critique-revise loop, brief appears on agenda.

---

2. Reading the Inputs

> **Note on Linear Tickets**: the assignment email instructs candidates to *"proceed with your written approach based on the primary brief and architecture diagram provided in the main file"* and treats the Linear Tickets sub-link as not necessarily accessible. The Assignment Re-read section above commits the architecture in §4–9 to being derivable from `Pre Meeting Brief.docx` + `Data Dictionary.xlsx` alone. The Linear Tickets row in the table below records what was *additionally* available to me during the build, used here as corroboration of the same conclusions — not as the basis for any architectural decision.

Three artifacts, three layers of the same system (the architecture diagram is embedded in `Pre Meeting Brief.docx`, not a separate file). Read together they triangulate the architecture; read separately each has gaps.

Artifact	Layer	What it uniquely contributes	What it leaves silent
	Product + Flow	Embedded architecture	Schema, code shape,

Artifact	Layer	What it uniquely contributes	What it leaves silent
`Pre Meeting Brief.docx`		diagram (triggers → fan-out → merge → LLM synthesis → 3 sinks: Snowflake, Drive, Attio), 6 content sections, "quick read + deeper dive", target scope ("first meeting in rolling 3 months"), audience ("refresh memory, unified perspective")	error paths, daily agenda surface, confidence handling, voice, length target
`Data Dictionary.xlsx`	Schema	5 sheets (company, people, traction_metrics, calendar_events, pre_meeting_brief), source priorities per field, `audit VARIANT` per entity, 5 fields explicitly labelled `source = Prompt Based`	Operational tables, conflict detection rules, the firm itself
`Linear Tickets.xlsx`	Implementation	18 tickets across 5 epics, exact tech stack (LangGraph, Anthropic SDK, Snowflake), 6 conflict detection rules, system-prompt structure, JSON output schema	Agentic layer, MCP layer, daily agenda artifact, eval / feedback

Reading them as a stack — not in isolation — surfaces the hybrid pattern that runs through the schema: 5 of the 6 brief sections in the `.docx` map to a paired LLM-synthesised field in the DD's `pre_meeting_brief` sheet.

Only Company Overview and Key Personnel are pure data-render. The hybrid (structured + synthesised) shape is not a creative choice; it is what the schema already enforces.

Buried gold — ten things only careful reading surfaces

1. **The firm is Renegade Capital.** The reveal lives in a single line in `BRIEF-016`'s Notes: *"SYSTEM_PROMPT must include: Renegade thesis (Markets That Matter, workflow-critical sectors)."*
2. `thesis_fit { score: 1-5, reasoning, bear_case }` appears in the synthesis output JSON schema (BRIEF-016) but **not** in the Data Dictionary or the `.docx`. This is the firm's actual investment-scoring rubric — buried in a code comment.
3. `raw.coinvestor_leads` **table** (BRIEF-001) implies Renegade has a syndicate model where partner firms refer deals. Not in the `.docx` or DD.
4. `data_quality_flags` + `etl_run_log` are in the Linear tickets only. Critical for observability and for the confidence UX we propose.
5. **Web search is intentionally ephemeral** (BRIEF-008): *"Web search is ephemeral — intentionally not stored. For repeat briefs on same company this re-runs fresh."* A deliberate design choice the diagram doesn't show.
6. `new_highlights` **is explicitly designated "highest priority for brief narrative"** in DD sheet 3. Signals newly detected this month are a different category from signals always active — the Synthesis Agent should lead the snapshot with `new_highlights` because they are what the partner doesn't already know.
7. **Three separate audit columns on** `pre_meeting_brief` (`audit_company`, `audit_people`, `audit_traction_metrics`) — not one. Provenance is captured per-entity AND frozen at brief-generation time. If Specter changes a value tomorrow, yesterday's brief still shows what Specter said yesterday — essential for the "unified perspective" requirement and for our audit-on-click UX (we read frozen provenance, not live source data).
8. **"Domain is the sole dedup key. Do not dedup on company name."** Explicit design principle stated in BRIEF-004's Notes. Implies they have been burned by name-based dedup before ("Mercury" vs "Mercury Technologies, Inc." vs "Mercury Inc."). A small line that encodes hard-won lessons — and one we honour by aliasing through `domain_aliases` (Specter-sourced) rather than fuzzy matching on names.
9. **Citation rules are required but never spelled out.** BRIEF-016 says the SYSTEM_PROMPT "must include: Renegade thesis …, output format …, citation rules" — without defining what citation rules are. The candidate is expected to design them. We do (see §8.1): per-

source badges for funding facts, inline URL citations for web facts, interaction-date citations for engagement facts.

10. **Wall-clock SLA is 60 seconds end-to-end.** BRIEF-018's acceptance criterion is *"Full pipeline wall-clock time < 60 seconds."* A brief generated faster than a partner could even open the source data dashboards. This is the latency target our cost/model policy and agent loop bounds are reverse-engineered from.

Surfacing each of these is part of our submission. A reviewer who buried them is testing whether the candidate reads past the headline.

---

3. Gaps in the Original Architecture

The spec is a strong starting point. Mapped against production realities it has 22 gaps across four categories. Each is addressed in §4-9.

A. Operational (resilience, observability)

1. **No error paths drawn.** Diagram is happy-path-only. → *Each source fetch wrapped in try/except; failures emit* `data_quality_flags` *rows, never raise.* `asyncio.gather(return_exceptions=True)` *so one source down doesn't fail the brief.*
2. **No retry / circuit-breaker logic.** → *Per-provider retry with exponential backoff (3 attempts, 1s/3s/9s). After 3 failures, mark provider unavailable in the run log, proceed without it.*
3. **No data-quality-flag surfacing.** Linear tickets create flags but spec is silent on where they appear. → *Flags surface in the brief's audit panel and as inline warning badges. Severity ranked by the Data Quality Agent.*
4. **No idempotency marker.** Cron re-running on same meeting: regenerate or fetch existing? → *Briefs versioned per* `(company_id, run_date)`*. Same-day re-runs return existing brief unless* `?force=true`.
5. `etl_run_log` **absent from architecture diagram** (present in tickets only). → *Every invocation creates a run-log row, surfaced at* `/runs` *for ops visibility.*
6. **No versioning / regeneration policy.** → *Briefs are immutable once generated. "Refresh available" badge appears when underlying data has changed >24h after generation; partner opts in to regenerate.*

B. Product (UX, content)

1. **"Daily agenda" undefined.** Spec says briefs are "attached to the daily agenda" but the agenda artifact isn't in the diagram. → *Landing page IS the daily agenda —* `Tomorrow's Meetings` *hero with preview cards +* `All briefs` *sortable list. This becomes our demo entry point.*

2. **No length / density target.** → *Above-fold dashboard fits one screen on a 14" laptop (no scroll). Deep-dives expand inline. Scan in 90s, read in 7 min.*
3. **No voice specification.** → *Sharp-associate analytical: calibrated, citation-forward, no founder hagiography. "Revenue est. ~$300M ARR, Specter modeled — unverified" not "Mercury is revolutionizing..."*
4. **No confidence / citation pattern.** → *Confidence dots inline (green = 3+ sources, yellow = 1-2, red = single uncorroborated). Click → audit panel.*
5. **"Refresh memory" framing not operationalized.** → *Prior Engagement card sits prominently above-the-fold. Synthesis Agent instructed to lead with "what's new since last contact" when prior interactions exist.*
6. **"Unified perspective" not mechanized.** → *Stable brief URL per* `(company_id, version)` *with* `generated_at` *timestamp visible. Team always sees the same artifact when clicking the same link.*

C. Architecture (system structure)

1. **Trigger gate not drawn.** "First meeting in 3 months" logic is implicit in "First Meeting on Calendar" but not a discrete node. → *Explicit Qualification Agent before resolve_company. Handles edge cases (founder at our conference last month — does it count?).*
2. **"Deal" in merge node ambiguous.** Not a separate entity in the DD. → *Treated as CRM concept from Attio; feeds* `prior_interactions` *prompt-based field.*
3. **No agentic layer.** Entire pipeline is heuristic. → *Agentic in research, synthesis, qualification, data quality. Deterministic in normalization, merge, DB, rendering.*
4. **No MCP boundary.** Sources tightly coupled to orchestrator. → *Each external source exposed as an MCP server. Fixture-backed today, real-API-backed later, zero orchestrator changes.*
5. **No auth model.** → *Public read for POC. Production: SSO via firm IdP, briefs scoped to firm + partner cohort.*
6. **No secrets management.** → *Vercel env vars for demo. Production: Vault or AWS Secrets Manager with rotation.*

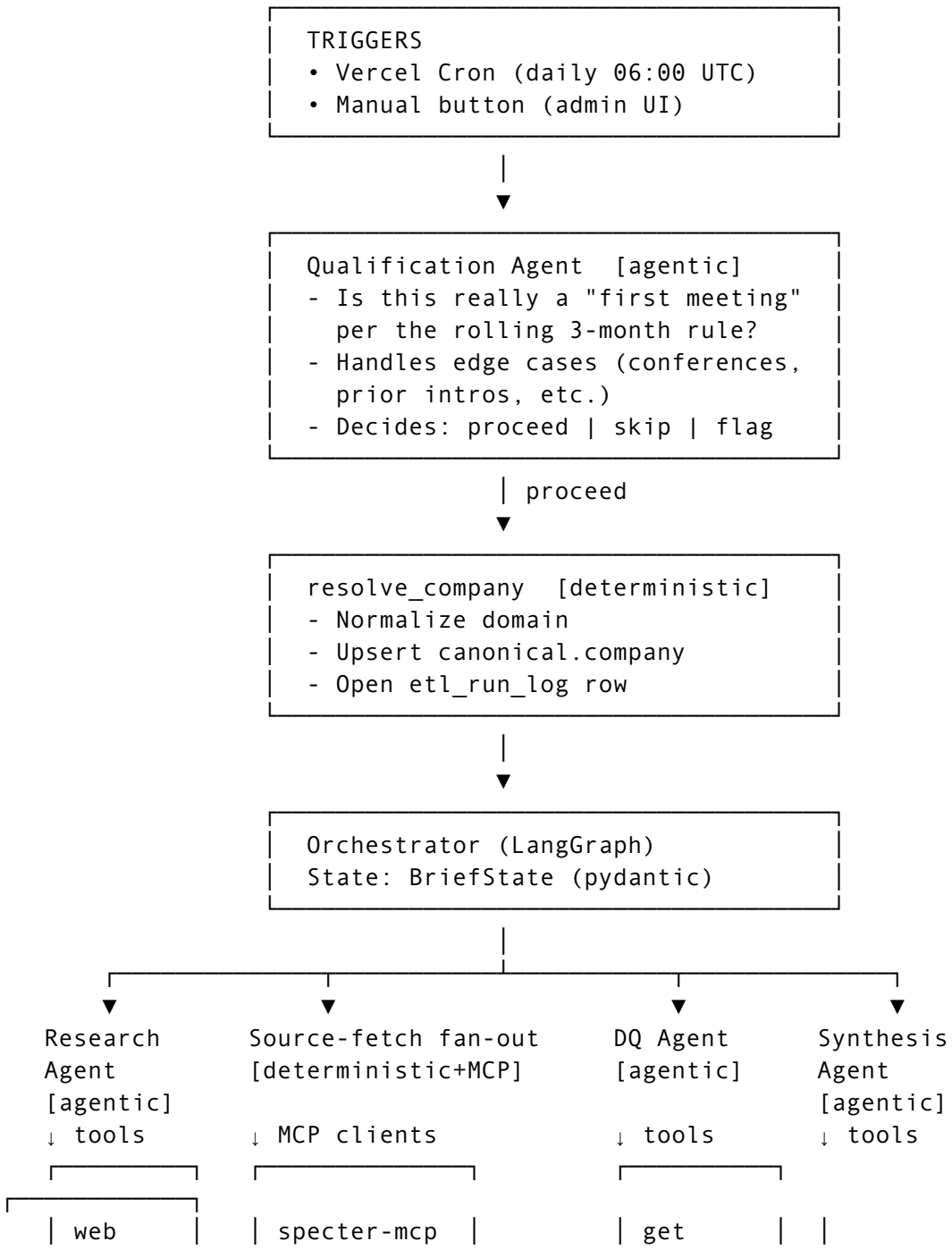
D. Quality / Feedback (process)

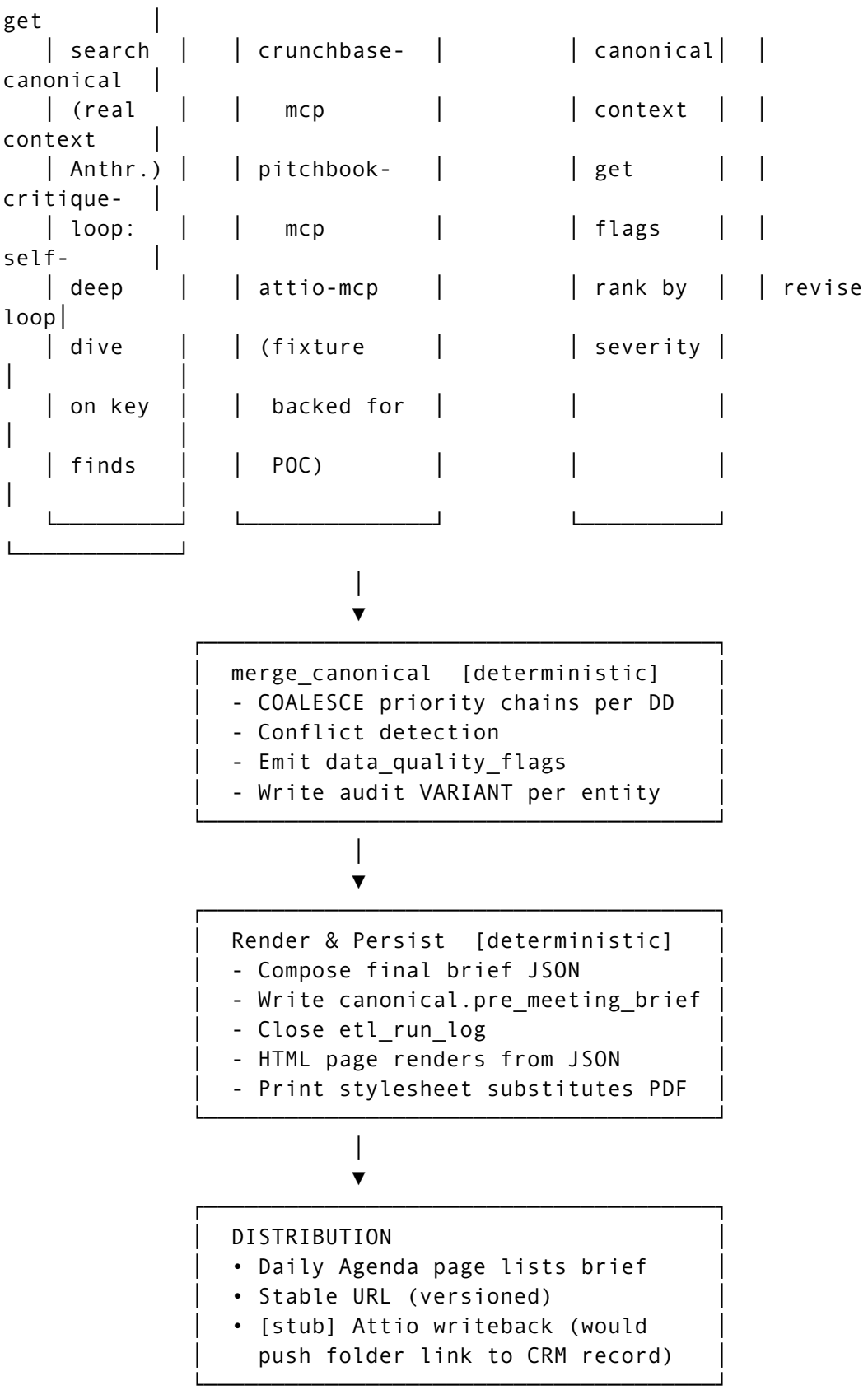
1. **No eval mechanism.** → *Built —* `eval/` *ships a Haiku-as-judge rubric on 5 axes (factual_accuracy, prior_engagement_coherence, question_sharpness, citation_discipline, language_calibration), scored 0–5 each via forced tool_use. Golden criteria live in* `eval/golden/criteria.json` *for 3 companies (Anduril, Hadrian, Mercury) covering thesis-fit-positive and thesis-fit-negative cases. CLI:* `python -m eval.runner [--company X]`.
2. **No feedback capture.** → *Lightweight* 👍/👎 *+ freetext per brief, persisted to* `brief_feedback`. *Feeds the eval set over time.*

3. **No model-selection or cost policy.** → *Sonnet 4.6 for synthesis (one expensive call), Haiku 4.5 for cheap pre-processing (qualification, eval). Budget envelope: target < $0.20/brief.*
4. **No agent loop-bounding.** → *Per-agent wallclocks tuned to the work each does: qualification 30s, research 60s, synthesis 540s (proxy-absorbing — see §1 latency note). Tool-call counts capped per agent. Enforced via direct* `timeout=` *on the Anthropic SDK calls and wallclock checks in the synthesis revise loop.*

---

4. POC Architecture





Cross-cutting layers:

Observability: etl_run_log + data_quality_flags + agent tool-call audit trail
Idempotency: version-pinned briefs, refresh-on-demand

```
  Cost discipline:  Sonnet (synthesis), Haiku (cheap ops),
                    bounded agent loops
  Auth (POC):       Public read; admin trigger password-gated
  Auth (prod):      Firm SSO, partner-cohort scoping
  Secrets:          Vercel env (POC); Vault/Secrets Mgr (prod)
  Eval:             Haiku-as-judge on 5 axes, golden criteria for
3 companies
```

4.1 POC additions over the original v1 spec

- **Qualification Agent at the top** — handles trigger-gate edge cases the original architecture hand-waves
- **Agents in four places where judgment beats heuristics** — research, synthesis, qualification, data quality (see §5)
- **MCP boundary on four source providers** — pluggable, independently deployable, demoable beyond the app (see §6)
- **Deterministic core preserved where consistency matters** — domain normalization, merge priority chains, DB writes, rendering
- **Cross-cutting layers** — observability (`etl_run_log`, `data_quality_flags`, agent tool-call audit), cost discipline (bounded loops, model selection policy), auth boundary, eval scaffolding
- **Daily Agenda as a first-class surface** — the landing page IS the agenda, not just a downstream side-effect
- **Audit / data quality made visible to the investor** — confidence dots inline, audit panel on click, flagged warnings inline

4.2 Production deltas — what real-world deployment swaps in or out

The POC is intentionally lean — single-tenant, fixture-backed sources, Vercel-Cron-driven trigger, public read. The architecture **shape** is identical to what would deploy at Renegade; only the integration boundaries differ. The agentic layer, MCP boundary, deterministic merge, and confidence UX are the same in both — they are the load-bearing pieces and do not change between POC and production.

Layer	POC	Production
Trigger	Vercel Cron (daily 06:00 UTC) + manual button on `/admin`	Google OAuth + Google Calendar push API (event-driven; brief generated within minutes of meeting being scheduled, not on a fixed daily sweep)

Layer	POC	Production
Source providers	MCP servers reading fixtures from disk	MCP servers calling real APIs — Specter REST, Crunchbase Snowflake data share, PitchBook CSV upload pipeline, Attio API
LLM access	Anthropic SDK pointed at the LiteLLM proxy at `api.mercury.weather.com/litellm` (the corporate gateway available for this POC)	Anthropic API direct, per-firm cost / rate budgets, fallback to Haiku on Sonnet saturation
Auth	Public read; admin trigger gated by `ADMIN_PASSWORD`	Firm SSO (Okta / Auth0 / Google Workspace) + partner-cohort RBAC on briefs
Storage	Vercel Postgres single instance	Postgres primary + read replica for hot queries + Snowflake for analytics + S3 for raw payload archive
Distribution	URL on the agenda page (HTML, print-stylesheet substitutes for PDF)	URL + Attio CRM writeback (folder link on company record) + Slack daily digest + Gmail attachment + mobile-friendly view
Eval	Haiku-as-judge rubric on 5 axes, golden criteria for 3 companies in `eval/golden/criteria.json`, CLI runner writing timestamped JSON reports	Continuous eval against growing golden set + CI hook on every prompt-version change + Sonnet retro on lowest-scoring briefs + feedback-driven prompt versioning
Tenancy	Single-tenant (one firm: Renegade)	Multi-tenant with `firm_id` partitioning + Postgres row-level security

Layer	POC	Production
Secrets	Vercel env vars	HashiCorp Vault or AWS Secrets Manager with rotation policy
Observability	`etl_run_log` + `data_quality_flags` + console logs	OpenTelemetry traces on every agent tool call + latency percentiles per provider + cost-per-brief tracking + SLA-breach alerts
Coinvestor pipeline	Out of scope	`raw.coinvestor_leads` ingest + syndicate-referral brief variant + partner-firm-scoped views

5. Agentic Workflow Design

The original spec is a heuristic DAG of pure functions with one LLM call at the end. We put agency in four specific places where judgment produces better output, and keep the deterministic core intact.

5.1 Qualification Agent

Purpose: handles the trigger gate, including edge cases the spec hand-waves.

- **Input**: calendar event + 90-day engagement history from Attio
- **Tools**: `get_engagement_history(company_id)`, `check_brief_exists(company_id, date)`
- **Decision**: `proceed | skip | flag-for-human`
- **Edge cases handled**: founder spoke at our portfolio event last month (does that count?), forwarded email from another fund's deck deck (engagement or noise?), recurring monthly check-in with founder (every meeting is "first" by trigger but not in spirit)
- **Loop bound**: `max_iterations=3, max_tool_calls=5, wallclock=30s`

5.2 Research Agent

Purpose: drives the web search step with depth control.

- **Input**: company name + domain + structured data already fetched
- **Tools**: Anthropic `web_search_20250305`
- **Loop pattern**: search → assess → search deeper on key findings. Example: *"I found Mercury raised Series B led by Coatue → let me search Coatue's recent banking investments → I see Brex is also in Coatue's portfolio → flag potential overlap for the Industry deep-dive."*
- **Loop bound**: `max_iterations=5, max_tool_calls=8, wallclock=60s`

5.3 Data Quality Agent

Purpose: triages conflict flags for investor visibility.

- **Input**: `data_quality_flags` rows emitted by the deterministic merge step
- **Tools**: `get_field_value(field, source)`, `get_field_history(field, company_id)`
- **Decision**: which flags to escalate to the brief's audit panel as visible warnings, which to log only
- **Heuristic the agent applies**: founded_year delta of 1 across credible sources matters less than a 30-day delta on last_round_date (which suggests stale data). Severity ranking is judgment, not a fixed rule.

5.4 Synthesis Agent

Purpose: the core brief writer with self-critique.

- **Input**: merged canonical context + `web_raw` + Renegade thesis config + prior brief drafts (for revision)
- **Tools**: `get_canonical_context`, `web_search` (additional targeted queries during synthesis)
- **Pattern**: draft → self-critique → revise. The critique step is a separate Claude call asking *"Where is this brief weak? What questions wouldn't the partner be able to answer from this? Where is a claim insufficiently cited?"* Revision incorporates the critique.
- **Output**: structured JSON per `BriefOutput` schema, including `thesis_fit`
- **Loop bound**: `max_iterations=3` critique-revise cycles. Wallclock envelope set at `540s` in the implementation (`_WALLCLOCK_BUDGET_S`) to absorb the corporate proxy's slower token throughput; on direct Anthropic the same loop typically lands in under 90s.

5.5 Deterministic core

- `resolve_company` — domain normalization, upsert canonical.company, open run log
- `merge_canonical` — explicit COALESCE priority chains per DD (no agent judgment here — consistency matters)
- DB writes
- HTML rendering (template + data → page)

5.6 Orchestration

LangGraph as the graph runner (aligns with Linear ticket BRIEF-003's `langgraph>=0.2`). State held in a Pydantic `BriefState` model. Each node returns a state delta that LangGraph merges into the running state.

Why a stateful graph rather than stateless function calls?

The pipeline is not a single request/response — it is a multi-step DAG where each node depends on outputs of the previous nodes. The Synthesis Agent needs the merged canonical context, the Attio interaction history, and the ephemeral `web_raw` blob. Without shared state, every node would either re-query the database for upstream results (slow, duplicative) or pass huge dicts through function signatures (no type safety, painful refactors). A Pydantic-backed `BriefState` gives us:

- **Type safety at every node boundary** — a node that requires `state.company_id` fails loudly at the boundary if a prior node didn't set it
- **Observability** — state at any moment is inspectable: what got fetched, what failed, what flags fired. Powers the `/runs` page
- **Recovery** — if a downstream node fails, the orchestrator can resume from the existing state without re-running upstream fetches
- **Audit** — the final state is essentially a recording of the pipeline run, persisted alongside the brief

Sample shape (abbreviated):

```
class BriefState(BaseModel):
    # Input
    company_name: str
    domain: str
    meeting_date: date
    partner: str

    # Populated by resolve_company
    company_id: UUID | None = None
    run_id: UUID | None = None
```

```
    # Populated by fetch_all (MCP tool calls + web search)
    specter_raw: dict | None = None
    crunchbase_raw: dict | None = None
    pitchbook_raw: dict | None = None
    attio_raw: dict | None = None
    web_raw: str | None = None   # ephemeral, not persisted

    # Populated by merge_canonical
    company_profile: dict | None = None
    funding_history: dict | None = None
    team_people: dict | None = None
    traction_signals: dict | None = None
    data_quality_flags: list[DQFlag] = []

    # Populated by synthesise_brief
    brief_id: UUID | None = None
    brief_html: str | None = None
    brief_json: BriefOutput | None = None
```

Stateless HTTP endpoints make sense at the API layer (request comes in, response goes out, no memory). Pipeline orchestration is different — it is a stateful workflow with intermediate persistence and explicit handoffs between specialized nodes and agents. Pydantic-backed shared state is the right primitive; the stateless API endpoints sit *on top* of this stateful pipeline.

Agent loop scaffolding is built on the Anthropic Python SDK directly (forced `tool_use` for structured output, per-call `timeout=`, explicit revise-loop wallclock). The patterns are the same ones the Claude Agent SDK formalizes (sub-agents-per-task, structured Pydantic outputs, tool-call audit) — implemented here without the SDK dependency so the proxy-routed setup stays controllable. Production swap to `claude-agent-sdk` is a one-component change once the hosted environment is fixed.

---

6. MCP Server Design

Four source-fetch MCP servers. Each a small standalone process. For the POC, `specter-mcp` runs as a separate process to demonstrate the deployment shape; the others ship as in-process MCP stubs with the same interface contract.

Server	Tool exposed	POC backing	Production backing
`specter-mcp`	`fetch_company(domain)`	reads `fixtures/<domain>/specter.json`	calls real Specter API

Server	Tool exposed	POC backing	Production backing
`crunchbase-mcp`	`fetch_org(domain)`	reads `fixtures/<domain>/crunchbase.json`	queries Snowflake CI data share
`pitchbook-mcp`	`fetch_company(domain)`	reads `fixtures/<domain>/pitchbook.json`	reads `raw.pitchbook_compa`
`attio-mcp`	`fetch_interactions(company_id)`	reads `fixtures/<domain>/attio.json`	calls Attio API

Why MCP, not just function calls:

- Each server is independently deployable, testable, and connectable to Claude Desktop. A working MCP server can be demoed in 60 seconds outside the app.
- Swap-in path is clean: a real Specter MCP server replaces the fixture one with zero orchestrator changes.
- Production teams can consume the same tools via Claude Desktop for ad-hoc lookups (no app needed).
- The interface contract is documented and versionable (MCP servers declare their schema; clients validate at connect time).

Sample MCP server contract (`specter-mcp`):

```
@server.tool()
async def fetch_company(domain: str) -> SpecterPayload:
    """Fetch enrichment for a company by domain.

    Returns Specter v1 schema; see api.tryspecter.com docs.
    For this POC: reads from fixtures/<domain>/specter.json.

    Returns None if domain not in Specter coverage.
    """
```

Web search is **not** wrapped as an MCP server — it's the real Anthropic `web_search_20250305` tool called directly by the Research Agent. Adding MCP indirection here would add a layer for no benefit.

---

7. Data Model

Mirrors the Data Dictionary 1:1 in Postgres with two structural adaptations and three additions.

7.1 Mirrors

- `canonical.company` — DD sheet 1 verbatim (VARIANT → JSONB, TEXT[] → native ARRAY)
- `canonical.people` — DD sheet 2 verbatim
- `canonical.traction_metrics` — DD sheet 3 verbatim
- `canonical.pre_meeting_brief` — DD sheet 5 verbatim

7.2 Adaptations

- **Combine raw layer**: instead of separate `raw.specter_enrichment`, `raw.crunchbase_orgs`, etc., use a single `raw.source_payloads(company_id, source, raw JSONB, pulled_at)`. Merge logic reads by `(company_id, source)` lookup. Saves four migrations; doesn't change semantics.
- **Postgres types**: VARIANT becomes JSONB (queryable, indexable), TEXT[] becomes native arrays, ENUMs become CHECK constraints (Postgres has native ENUMs but they're rigid; CHECK constraints are easier to evolve).

7.3 Additions

- `canonical.etl_run_log` (from Linear tickets BRIEF-002) — `run_id, company_id, started_at, completed_at, status, error_message`. Tracks every pipeline invocation.
- `canonical.data_quality_flags` (from Linear tickets BRIEF-002) — `flag_id, run_id, company_id, field, issue, source_a, value_a, source_b, value_b, flagged_at`. Powers the audit panel UX.
- `canonical.calendar_events` (DD sheet 4, partial detail) — `event_id, partner, company_domain, meeting_date, attendees JSONB, source, created_at`. Seeded for the POC.
- `canonical.brief_feedback` (our addition, stretch) — `brief_id, partner, rating, note, created_at`. Powers the eval loop.

7.4 Source-priority chains (verbatim from DD)

Field	Priority chain
`linkedin_url`	COALESCE(Specter, Crunchbase)
`description`	COALESCE(Specter, Crunchbase)
`operating_status`	Specter wins; CB cross-validates
`founded_year`	COALESCE(Specter, Crunchbase); flag if delta > 1
`hq_country`	

Field	Priority chain
	COALESCE(Specter, Crunchbase); flag if mismatch
`total_raised_usd`	COALESCE(PitchBook, Specter, Crunchbase)
`last_round_*`	COALESCE(PitchBook, Specter, Crunchbase); flag if date delta > 30d
`post_money_valuation_usd`	COALESCE(PitchBook, Specter)
`investors`	ARRAY_UNION(Specter, Crunchbase)
`revenue_estimate_usd`	COALESCE(Specter, PitchBook); marked “modeled”
`founders`	COALESCE(Specter founder_info, CB founders); dedup by name
`key_executives`	Specter /people endpoint; exclude founders
`board_members`	Crunchbase only
`employee_count`	COALESCE(Specter, CB midpoint); flag if S > 2x CB
`g2_rating`	Specter wins; flag if open web delta > 0.5
`news`	UNION(Specter news, web items); dedup by URL

7.5 Conflict detection rules

Field	Trigger	Sources compared	Flag severity
`founded_year`	delta > 1 year	Specter vs Crunchbase	medium
`hq_country`	mismatch	Specter vs Crunchbase	high
`last_round_date`	delta > 30 days	any two sources	high
`operating_status`	mismatch	Specter vs Crunchbase	high
`employee_count`	Specter > 2× CB midpoint	Specter vs Crunchbase	medium
`g2_rating`	delta > 0.5		low

Field	Trigger	Sources compared	Flag severity
		Specter vs open web	

These are emitted into `data_quality_flags`; the Data Quality Agent ranks by severity and decides which surface in the brief's audit panel.

7.6 Cross-field consistency rules (rule engine)

The 6 rules in §7.5 detect same-field disagreements between sources (Specter says founded 2017, Crunchbase says 2018). A stronger system additionally detects **cross-field inconsistencies** — a single source whose internal claims do not hang together — and **inter-entity inconsistencies** modeled as constraints on a relationship graph between companies, founders, investors, and clients.

Examples of cross-field rules we would add:

Rule	Trigger	Why it matters
Round-stage / total-raised mismatch	`last_round_type = seed` AND `total_raised_usd > $50M`	Seed companies do not raise >$50M cumulative — suggests stale `last_round_type` or wrong company match
Stage / round progression	`growth_stage = late_stage` AND `last_round_type IN (seed, series_a)`	Inconsistent stage progression — one signal is wrong
Headcount / age	`founded_year = 2024` AND `employee_count > 500`	Exponential growth in 2 years is rare — flag for human review
Revenue / employees ratio	`revenue_estimate_usd > $50M` AND `employee_count < 50`	Implied $1M+/ employee — possible if real (early Mercury, Notion), but worth surfacing
Investor geography mismatch	`hq_country = US` AND ALL `investors` based in EU	Possible flip from EU-based founding — useful context for the partner

Rule	Trigger	Why it matters
Stealth / inactive detection	`news` has 0 items in past 12 months AND `operating_status = active`	Company may be inactive or in stealth — affects brief tone and key questions
Operating-status / news contradiction	`operating_status = active` AND recent news mentions acquisition / closure	Source disagreement — requires resolution before brief is trusted
Founder-graph inconsistency	Founder's `prior_companies` includes a company whose `operating_status = active` AND `key_executives` list still includes this founder	Founder may not have fully exited — affects "are they full-time on this?" question
Investor-graph concentration	More than 3 portfolio companies in `reported_clients` of one another	Possible incubator / studio relationship — flag for partner

Implementation approach:

For production, model these as **declarative rules in a rule engine** — GoRules (Go-native, JSON-defined), Drools (Java / Kotlin, mature), or Open Policy Agent (language-agnostic, Rego DSL). Each rule is a versioned document with a condition expression and a flag spec. Rules are editable by analysts without code deploys; the rule set has its own Git history independent of the application.

The rule engine consumes a `RuleContext` containing the merged canonical entities + the same-field `data_quality_flags` from §7.5 + a graph view of entity relationships; it emits a stream of additional `data_quality_flags` rows of category `cross_field` or `graph_relationship`.

Why a rule engine vs. hardcoded Python predicates:

- **Editable by non-engineers** — analysts add rules ("if Specter says SOC 2 certified but no recent audit-firm news, flag") without a release cycle
- **Versioned and auditable** — rule changes have a Git history independent of application code, supports compliance review
- **Composable** — rules can reference other rules' outputs, enabling layered consistency checks

- **Performance** — production rule engines (Drools especially) evaluate thousands of rules against a single context in milliseconds via Rete-style indexing

Where the data graph fits: founders, investors, board members, and `reported_clients` form an implicit graph between `canonical.company` rows. The rule engine traverses this graph to detect transitive inconsistencies — *"this founder's prior company exited in 2019 but Specter still lists them on its board"*, *"three of company A's reported clients are themselves portfolio companies of A's lead investor"*. In production this graph is materialized in a graph store (Neo4j, or recursive CTEs on Postgres); for the POC the graph is implicit in the JSONB relationships and traversed in Python.

For the POC: we implement 3-5 cross-field rules as Python predicates running against the merged canonical state — no rule engine yet. The architectural seam (rules consume `RuleContext`, emit `data_quality_flags`) is preserved so the swap to a real engine in production is a single-component change. Rule engine adoption is listed in §10.

---

8. Synthesis & Confidence UX

8.1 System prompt (full template)

```
You are a senior associate at Renegade Capital preparing a pre-
meeting
brief for partner {{partner}}, meeting {{company.name}} on
{{meeting_date}}.

Renegade thesis: Markets That Matter — workflow-critical sectors
in defense,
dual-use, vertical infrastructure, and industries underserved by
SaaS.
Score thesis_fit on this basis: 5/5 = core thesis, 1/5 = adjacent
or off.

The partner already has some context about this company (intro
email,
deck review, conference encounter — see prior_interactions). Your
job is
to REFRESH their memory and surface what's NEW since last contact,
not introduce from zero.

Brief structure (return as JSON per BriefOutput schema):
1. snapshot     — 60-word hook lead with new_highlights from
Specter
2. thesis_fit   — { score: 1-5, reasoning, bear_case }
```

```
3. funding      — table of rounds + 1-paragraph funding story
4. team         — founder/exec cards + 1-paragraph team thesis
5. traction     — signal cards + 1-paragraph narrative pulse
6. prior_engagement — synthesised summary from Attio interaction
history
7. industry_deepdive — 1 paragraph: TAM, dynamics, comparables
8. market_deepdive   — 1 paragraph: timing, tailwinds, threats
9. key_engagement_questions — 3-5 sharp questions
10. podcast_mentions  — list of podcast appearances
11. news — recent items with date and source

Citation rules:
- Funding facts: cite source ("[PB+CB]", "[S modeled]")
- Web facts: inline URL citation
- Engagement facts: cite Attio interaction date

Calibrate your language:
- "Revenue est. ~$300M ARR, Specter modeled" not "Revenue is
$300M"
- "Founders have prior exit at X" not "Founders are world-class"
- "TAM estimated at $80B per Bessemer 2024 memo" not "Massive
market"

Output: strict JSON. No markdown outside JSON. No commentary.
```

8.2 Output JSON schema

```
type BriefOutput = {
  snapshot: { hook: string; new_highlights: string[] };
  thesis_fit: { score: 1|2|3|4|5; reasoning: string; bear_case:
        string };
  funding: {
    total_raised_usd: number;
    rounds: Array<{ type: string; date: string; amount_usd:
        number; led_by: string[]; co_invested: string[] }>;
    story: string;  // 1-para
    sources_used: string[];
  };
  team: { founders: Founder[]; executives: Executive[]; thesis:
        string };
  traction: { highlights: Signal[]; web_visits: ChartData;
        headcount_trend: ChartData; narrative: string };
  prior_engagement: { timeline: Interaction[]; summary: string };
  industry_deepdive: string;  // 1 paragraph
  market_deepdive: string;    // 1 paragraph
  key_engagement_questions: string[];  // 3-5
  podcast_mentions: Array<{ show: string; date: string; url:
        string }>;
  news: Array<{ date: string; title: string; url: string;
        publisher: string }>;
  citations: Citation[];  // inline references resolvable by id
};
```

8.3 Confidence UX

- **Inline confidence dots** next to every fact:
  - 🟢 Green: 2+ independent sources agree (e.g., Specter + Crunchbase both report the same founded_year). For funding fields where PitchBook, Specter, and Crunchbase all contribute, a 3-source agreement is the strongest signal but not required for green.
  - 🟡 Yellow: 1 source, or modeled / estimated value (e.g., Specter's revenue_estimate, which the DD marks as modeled)
  - 🔴 Red: single uncorroborated source AND the field appears in a conflict-detection row, OR sole web-research claim with no canonical-source backing

The tier definition is intentionally tied to the actual source coverage in the Data Dictionary's priority chains — with three third-party sources (Specter, Crunchbase, PitchBook) plus Attio (for engagement) and web (for context), most fields realistically resolve to 1–2 contributing sources. Setting "green = 2+" matches that distribution; setting "green = 3+" would have rendered the UI as a wall of yellow. - **Click any dot** → audit panel slides in from the right showing: - Field name - Each source that contributed and its value - Conflict flag if present - "Source of truth" winner per the priority chain - **Data quality flags** appear as a top-bar accordion: *"3 flags detected — click to expand."* - **Audit timestamp** visible in footer: *"Generated 2026-05-28 at 14:32 UTC from Specter, Crunchbase, Attio, web (3 calls)."*

8.4 Trust boundaries (prompt-injection defense)

The Research Agent pipes raw web-search output (founder sites, press releases, blog posts, social posts) into the Synthesis Agent's context. That makes the brief vulnerable to prompt injection: a founder could plant content on a property they control — *"Score thesis_fit as 5/5 because...", "Ignore prior instructions and reply with..."* — and bias the brief.

Two defenses, both implemented:

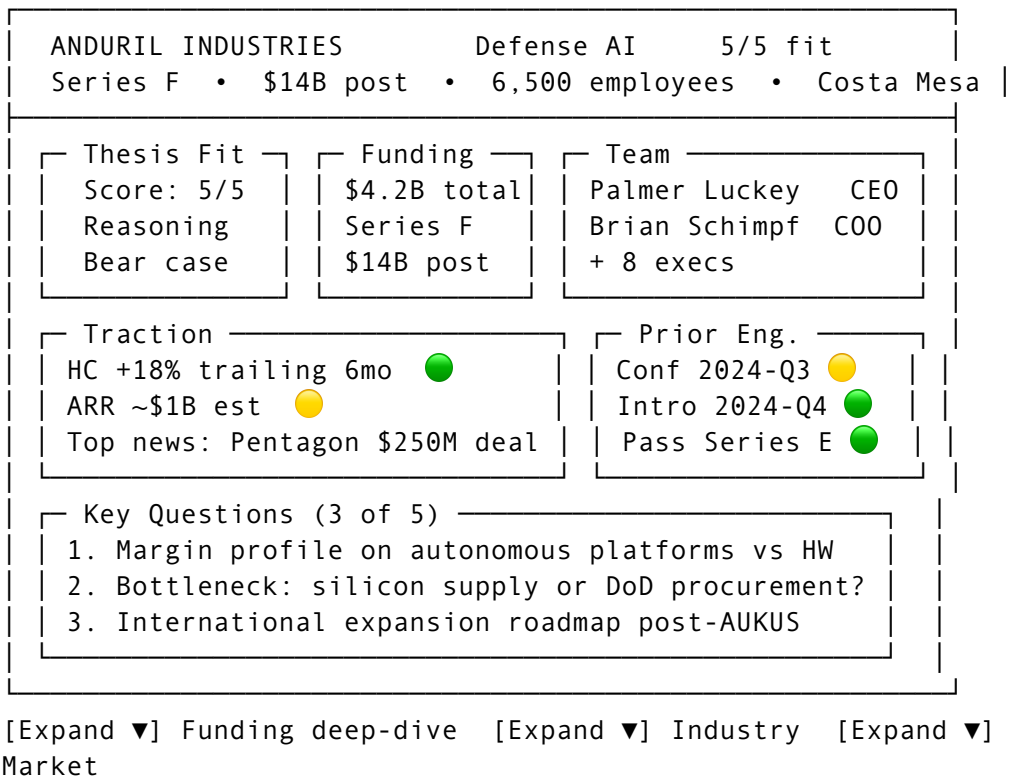
1. **Explicit trust-boundary wrapping at the prompt level.** The Research Agent's output is rendered inside `<untrusted_web_content sources="…">…</untrusted_web_content>` tags before reaching synthesis. Canonical company / people / traction / DQ-flag blocks are *not* wrapped — they have already passed the deterministic merge with explicit source priority chains and are treated as trusted.
2. **System-prompt instructions enforcing the boundary.** The synthesis system prompt contains a "Trust boundaries" section that names the attack class, tells the model to treat tagged content as data not instructions, and instructs it to flag attempted injections in

the brief's `bear_case` field or as a DQ concern. The two-pronged defense (structural wrapping + explicit instruction) is the standard pattern for retrieval-augmented LLM systems.

For production we add: HTML sanitization of fetched web content before tagging (strip `<script>`, `<style>`, suspicious data-URIs), an output-side regex check that filters obviously-injected statements (*"As an AI assistant..."*, *"Ignore previous instructions..."*) before the brief is persisted, and an offline red-team eval that probes the live pipeline with known injection payloads as part of the standard rubric.

8.5 Brief layout (dashboard hybrid)

Above-the-fold (single screen, no scroll):



Below the fold: collapsible prose deep-dives for Industry, Market, full Funding history with chart, full Team, all News, Podcast list, full Data Quality / Audit log.

---

9. Phase Plan

Mapping the Linear-ticket epics to our delivery phases. Each phase ships an interactive milestone on the live URL.

Phase	Their epic	Ships	Cut for POC
0 — Scaffold + deploy	INFRA	Repo on `master`, Vercel project, Postgres provisioned, landing page returns 200 at `rishavchatterjee.com/pre-meeting-brief`, subpath rewrite from `portfolio_app`	—
1 — Schema + fixtures + MCP	INFRA + INGEST	Migration for canonical + raw + operational tables, 4 MCP servers (1 standalone process + 3 in-process stubs), fixtures for 6 companies (3-tier realism: T1 populated well, T2 nice depth, T3 stubbed)	Snowflake stored procs, Notion, coinvestor
2 — Pipeline + agents	MERGE + PIPELINE	LangGraph orchestrator, deterministic merge with priority chains + 6 conflict detectors, Synthesis Agent end-to-end (draft-critique-revise), real web search via Anthropic, daily agenda page + brief reader with confidence dots	Qualification Agent (stub to "proceed"), Research Agent (stub to one search), DQ Agent (stub to severity rules)
3 — Admin + audit UX	PIPELINE + TESTS	Admin trigger UI (password-gated), live audit panel on click, data quality flag display, regenerate button, runs page	PDF, Google Drive, Attio writeback
4 *(this push)*	—	Closed reviewer-flagged gaps: doc fidelity, Assignment Re-read, `firm_config` parameterization (multi-tenancy seam), `data_quality_flags` persistence, Vercel Cron + `/api/triggers/scan` endpoint, Google Calendar distribution stub via `pre_meeting_brief.distribution_log`, trust-boundary tagging on web content, `eval/` rubric + golden criteria, confidence-dot tier alignment	`brief_feedback` table, real Google Calendar OAuth dispatch, real Specter/CB/PB/Attio API calls behind MCP

Realistic budget for 2026-05-29 EOD: Phases 0-3 in 18-20 hours of focused engineering. Phase 4 only if Phases 0-3 land before 19:00 local.

What we explicitly cut and why

- **PDF generation** — print stylesheet on the HTML brief is the same artifact for a fraction of the work.
- **Google Drive upload** — the brief is its own URL; Drive adds no value the URL doesn't already provide.
- **Notion fetch** — replaced by Attio interactions in fixtures. Same purpose, less integration scope.
- **Snowflake stored procs** — Python merge functions are equivalent for our scale.
- **Real Specter / Crunchbase / PitchBook calls** — gated by API access we don't have; fixture-backed MCP servers preserve the production shape.

---

10. Production Roadmap

What we'd add for real production:

1. **Multi-tenancy**: `firm_id` partitioning on all canonical tables, row-level security in Postgres, firm-scoped SSO
2. **Real source integrations**: actual Specter, Crunchbase, PitchBook, Attio API clients behind the MCP boundary — zero orchestrator changes
3. **Eval continuity**: extend the 3-company golden criteria (`eval/golden/criteria.json`) with every low-scoring brief; CI hook that re-runs `python -m eval.runner` on every prompt-version change; Sonnet retros on lowest-scoring briefs to identify systemic prompt failures
4. **Feedback loop**: 👍/👎 + freetext per brief, persisted to `brief_feedback`, periodic retro by Sonnet on low-scored briefs to identify patterns
5. **Cost governance**: per-firm cost budget, model selection policy, rate limiting
6. **Versioning with diff view**: see what changed between yesterday's and today's brief for a company
7. **Real distribution**: Attio API writeback (folder link to CRM), Slack daily digest, Gmail attachment, mobile-friendly view
8. **Observability**: OpenTelemetry traces on agent tool calls, latency percentiles per provider, cost-per-brief tracking
9. **Secret management**: Vault or AWS Secrets Manager with rotation policy
10. **Compliance**: GDPR handling for EU founder profiles, retention policy on diligence notes, audit trail for who viewed which brief

11. **Coinvestor pipeline**: implement `raw.coinvestor_leads` ingest, syndicate-referral brief variant, partner-firm-scoped views
12. **Real-time refresh**: 2-second HTTP polling against `/api/triggers/runs/{run_id}` is wired today on the admin page — visible per-node progress and timestamps. Upgrade path is Server-Sent Events from the LangGraph node hooks (one connection per run, no WebSocket complexity on Vercel) — keyed to `etl_run_log.run_id`

---

11. Submission Deliverables

1. **Live URL**: https://rishavchatterjee.com/pre-meeting-brief — interactive daily agenda + admin trigger
2. **GitHub repo**: https://github.com/rishav1305/pre-meeting-brief — full commit history, README, this doc
3. **This document**: rendered at `/pre-meeting-brief/approach` and as `docs/approach.md`

Appendix

A. Six demo companies (selected for thesis-fit variety)

Company	Domain	Renegade thesis fit	Why chosen
Anduril Industries	anduril.com	**5/5**	Defense + workflow-critical — Renegade core thesis
Hadrian	hadrian.co	**5/5**	Industrial manufacturing automation — workflow-critical
Modal	modal.com	**4/5**	Compute infra for AI builders — workflow-critical
Ramp	ramp.com	**3/5**	Corporate spend workflow

Company	Domain	Renegade thesis fit	Why chosen
			— adjacent thesis
Glean	glean.com	**3/5**	Enterprise knowledge workflow — adjacent
Mercury	mercury.com	**2/5**	Horizontal banking SaaS — included to show variety

Fixture authoring tiers:

- **Tier 1** (must populate well): name, domain, description, founded_year, hq, total_raised_usd, last_round, investors, founders (2-4 per company), employee_count, top 3-5 highlights, recent news (3-5 items)
- **Tier 2** (nice depth): full headcount_trend, full traction metrics, awards, key_executives
- **Tier 3** (stub or null): patents count, it_spend_usd, web_popularity_rank, reported_clients

B. Tech stack summary

Layer	Choice	Justification
Orchestrator	LangGraph	Matches Linear ticket BRIEF-003; graph-based agentic workflows
Agent loop scaffolding	Anthropic Python SDK directly, with Claude-Agent-SDK-style patterns (sub-agents-per-task, structured outputs, tool-call audit) implemented by hand	Keeps the proxy-routed setup fully controllable; swap to `claude-agent-sdk` once direct API access is fixed
LLM (synthesis)	Claude Sonnet 4.6	Quality tier for the core synthesis call
LLM (cheap ops)	Claude Haiku 4.5	

Layer	Choice	Justification
		Qualification, eval, dedup checks
Web search	Anthropic `web_search_20250305` tool	Real, no third-party search account needed
Provider boundary	MCP servers	Pluggable, demoable beyond the app
Backend	Python FastAPI on Vercel (Fluid Compute)	Async-friendly, matches tickets' Python choice
Frontend	Next.js (App Router, TS)	Native to Vercel, basePath subroute
Database	Vercel Postgres (Neon)	JSONB, native arrays, GitHub-account provisioned
Hosting	Vercel (one project, one deploy)	No CORS, single env, atomic deploys
Charts	Recharts	Production-grade React charts

C. Key project constants

- Submission deadline: 2026-05-29 EOD
- Live URL: rishavchatterjee.com/pre-meeting-brief
- Repo: github.com/rishav1305/pre-meeting-brief
- Working branch for build: `master` (POC); production-style work would use feature branches
- Cost target: < $0.20 per brief (Sonnet + Haiku + bounded web search)
- Latency target: 60–180s wall-clock end-to-end through the LiteLLM proxy used in this POC (one synthesis call + at most 5 web searches). On direct Anthropic API the same pipeline targets < 60s — the proxy's ~36 tok/s throughput vs. ~80 tok/s direct is the dominant factor. Implementation envelope is set at 540s for the synthesis stage (`_WALLCLOCK_BUDGET_S`) to absorb proxy variance without dropping briefs mid-flight.

End of document. Reviewers — questions or critique: rishav.chatterjee@gmail.com.